KMS Enterprise Knowledge Platform: Implementation Specification

Product	KMS (Knowledge Management System)
Document	Implementation Specification, Enterprise Knowledge Platform (EKP)
Version	1.0 (draft), 2026-10-11
Implements	`docs/ekp/KMS_EKP_PRODUCT_REQUIREMENTS_SPECIFICATION.md` (PRS): requirements E1–E32
Plan	`docs/ekp/EKP_IMPLEMENTATION_PLAN.md` : phases, workstreams, team workflow
Codebase baseline	`dev @ 57abc78` (2026-10-09)

1. Purpose

The PRS says **what** KMS must do. This document says **how**:

- the current architecture (as built) and what is already implemented;
- the shared foundations to build first;
- a technical design for each of the 32 requirements: data model, APIs, worker jobs, UI and tests.

Designs here are the **starting point**. Each workstream owner refines them in their `REQUIREMENTS_EKP_<WS>.md` during the requirements review (implementation plan §4). If a design changes, update this document in the same PR.

Status key: Met (done) · Partial (partly built) · Gap (not started)

2. Completion Status: What's Done Today

2.1 Scorecard

Status	Count	Requirements
Met	**8**	E6, E7, E8, E9, E17, E18, E20, E27
Partial	**18**	E1, E2, E3, E4, E5, E10, E11, E13, E19, E21, E22, E23, E24, E26, E28, E29, E30, E32
Gap	**6**	E12, E14, E15, E16, E25, E31

2.2 Completed requirements (Met)

#	Requirement	How it's implemented	Key files
E6	Recommendations and trending content	Personal recommendations ranked from hybrid search, user activity and answer feedback. "Trending in your department" from `DocumentInteraction`. Admin effectiveness view.	`src/lib/recommendations.js` , `src/lib/recommendationRankingPolicy.mjs` , `src/app/api/org/[orgId]/recommendations/`

#	Requirement	How it's implemented	Key files
E7	Natural-language answers	Org-wide streaming RAG chat (server-sent events) with a conversation list. *Remaining polish: show the answer on a unified search results page (see §5, E7).*	`src/app/api/org/[orgId]/chat/route.js`, `src/app/(app)/org/[orgId]/chat/page.jsx`
E8	Cited, permission-aware answers	Retrieval is permission-filtered in SQL **before** prompt construction. Typed citations (document / decision / lesson). Signed S3 URLs for sources. Every question and its cited docs logged in `ChatAuditLog`, including denied attempts.	`src/app/api/org/[orgId]/chat/route.js`, `src/lib/hybridSearch.js`, `src/lib/s3SignedUrl.js`
E9	Follow-up questions	Last 6 turns sent with each question. Active document/topic carried as session context. Entity extraction from the question. Cross-conversation topic memory (`OrgMemberMemory`).	`src/app/api/org/[orgId]/chat/route.js`, `src/lib/entityExtraction.js`, `src/lib/orgMemberMemory.js`
E17	Graph-enriched search, navigation, Q&A	`DocumentRelationship` scoring (Rank 12). Org knowledge graph with typed nodes and edges plus provenance (Rank 13). Search results expanded with related documents. Graph explorer page.	`src/lib/knowledgeContext.js`, `src/lib/knowledgeGraph.js`, `src/lib/relationshipScoringPolicy.mjs`, `src/app/(app)/org/[orgId]/knowledge-graph/`
E18	Drag-and-drop upload	Batch upload (up to 10 files) with category and department. Presigned S3 upload, then ingestion queued on SQS.	`src/components/repository/UploadToRepositoryModal.jsx`, `src/app/api/s3/upload/`, `src/app/api/documents/ingest/route.js`
E20	Auto-classification with human review	Worker classifies category and suggests department with confidence. Below thresholds (0.72 / 0.78), the document goes to the **Needs Review** queue. Re-runs on replace and on SharePoint update.	`worker/classify.js`, `src/app/api/org/[orgId]/needs-review/`, `src/app/(app)/org/[orgId]/needs-review/`
E27	Duplicate and near-duplicate detection	Content hash (exact) plus embedding similarity ≥ 0.92 (near) into `DocumentDuplicate`, with a review state. Contradiction detection into `DocumentConflict`.	`worker/classify.js` (`detectDocumentDuplicates`), `worker/detectConflicts.js`, `src/app/api/documents/[id]/duplicates/`

2.3 What already exists for the partly built requirements (Partial)

#	Already built	Still to build
E1	Org dashboard (stats + health panel)	Admin-configurable landing page, branding
E2	SharePoint connector: Graph app-only auth, Sites.Selected grant, delta sync, `SyncRun` history, AES-256-GCM token storage, site → department mapping	Per-file ACLs, scheduled sync, Content Server and ServiceNow connectors
E3	Hybrid vector (pgvector) + BM25, LLM query expansion, graph expansion	Ontology (term/synonym) expansion and boosting. One search engine for all UIs.
E4	Repository filters: department, category, lifecycle, file type, date	Taxonomy facets with counts, facets on search results
E5	Role/department-scoped discovery, department pages, expert topic pages, starred documents	Personal homepage, topic hubs, saved searches, watchlists, notifications

#	Already built	Still to build
E10	high/medium/low confidence (`computeConfidence`)	Numeric score, "why this answer" panel, feedback capture
E11	PDF (+ Tesseract OCR), DOCX, XLSX/XLS/CSV, TXT	PPTX, images, audio/video
E13	Full-text and vector indexing, in-app viewer	Thumbnails, image/slide/video previews
E19	AI structured summary, key points and reading guide, regenerate	Review state, edit/approve, audit
E21	Classification confidence gate	Configurable required-field rules, publish blocking
E22	Role-gated lifecycle state machine (`draft → published`)	Configurable multi-stage workflow engine
E23	`published → archived → retired` transitions, hard delete, in-place replace	Supersede, version history, soft delete, **audit logging**
E24	Implicit owner (uploader / department admin)	Owner and steward fields, bulk reassignment
E26	Stale-document CLI script (`scripts/task-8/flag-stale-documents.mjs`)	Scheduling, configurable rules, automatic review tasks
E28	Health panel: answer confidence, open conflicts, knowledge gaps	Recency, ownership, review, broken links, duplicates
E29	`ChatAuditLog` , `DocumentInteraction` (views + dwell), recommendation effectiveness	Search/click logging, adoption dashboard
E30	Knowledge-gap detection script (`scripts/task-5d/detect-knowledge-gaps.mjs`), dwell capture	No-results tracking, CTR@k, reformulation, quality dashboard
E32	Graph read access to SharePoint/OneDrive	Entra ID sign-in, deep links, no-copy storage mode

2.4 Beyond the 32 (already built, reusable)

Expert Discovery (Rank 9), Lessons Learned (Rank 11), Decision Intelligence with human review (Rank 14), Project Intelligence (Rank 15, in progress), Topic Evolution (Rank 16), Knowledge Gap detection, Conflict detection. Several of these are **reused** below. Examples: topic hubs (E5) reuse expert and topic data, and explainability (E10) reuses conflict flags.

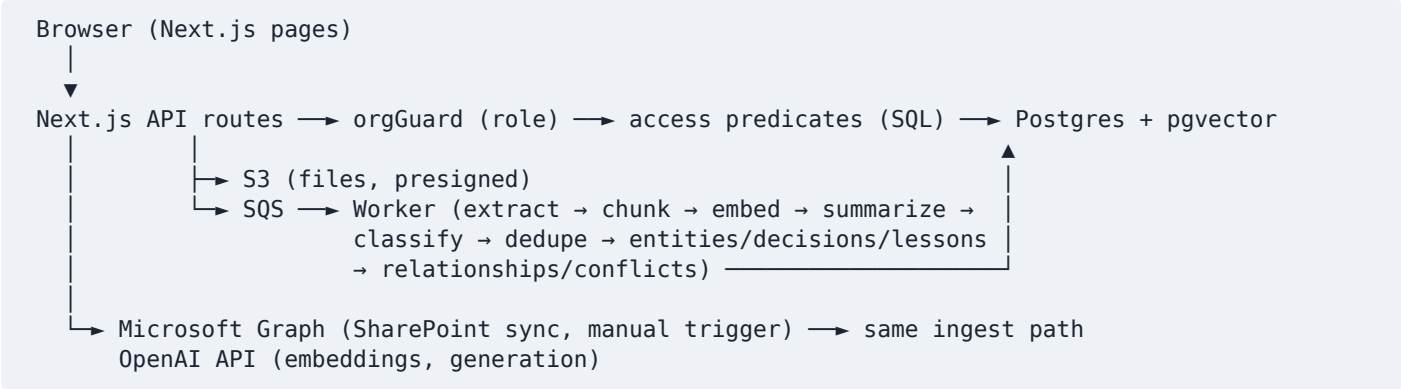
3. Current Architecture (As Built)

3.1 Stack

Layer	Technology
Web app and API	Next.js 16 (App Router), React 19, Tailwind 4, Headless UI, lucide icons
Auth	NextAuth v4, JWT sessions. Credentials and email OTP providers (`src/lib/auth.js`).
Database	PostgreSQL (Neon, AWS us-east-2) via Prisma 6. **pgvector** `vector(1536)` on `Chunk.embeddingVec` .
File storage	AWS S3 (`kms-document-storage` , us-east-2), presigned upload and download
Queue	AWS SQS (`kms-document-ingestion-queue-dev`)

Layer	Technology
Worker	Long-running Node process polling SQS (`worker/index.js`): extraction, OCR, chunking, embeddings, summary, classification, duplicates, entities, decisions, lessons, relationships, conflicts
AI	OpenAI API (per-org key): `text-embedding-3-small` for embeddings, `gpt-4o-mini` and chat models for generation
Email	nodemailer (`src/lib/mailer.js`)
Tests	`node --test` suites under `scripts/task-*` (static, logic and integration)

3.2 Component view



3.3 Key flows

- **Ingestion:** upload or sync → `POST /api/documents/ingest` creates a `Document` (status `pending`) → SQS message → worker processes it, writing `Chunk` , `Topic` / `TopicDocument` , classification fields, `DocumentDuplicate` , `Entity` , `Decision` , `Lesson` , `DocumentRelationship` and `DocumentConflict` → status `completed` .
- **Q&A:** question → `hybridOrgSearch` (permission-filtered vector + BM25 + graph expansion) plus decision and lesson evidence → prompt with grounded context and the last 6 turns → streamed answer, with sources and confidence → `OrgMessage` and `ChatAuditLog` .
- **Access control:** org role from `resolveOrgRole` (`super_admin` , `dept_admin` , `member` , `guest`). Document visibility = org admin, OR (published repository document AND (no department OR the user is a member of it)), OR an org-scope project document in the user's department, OR the user's own document.

3.4 Known technical debt that affects the EKP

#	Issue	Impact	Fix (see §4)
D1	**The document-access predicate is duplicated** in `vectorSearch.js` (3 copies), `knowledgeContext.js` (`accessSql`), `expertDiscoveryQuery.mjs` , `decisionIntelligence.js` , `lessonsIntelligence.js` , plus a JS version (`documentAccess.js`) and route-level checks	Per-file source permissions (E2) would have to be added in about 8 places, which risks leaks	**F1**
D2	**Two search paths.** The repository page uses its own embedding search with a text fallback (`api/org/[orgId]/repository`). `hybridOrgSearch` is used only by chat and recommendations. `api/org/[orgId]/search` has no UI consumer.	Ontology, facets and analytics would need building twice	**F7**
D3	**No scheduler.** Stale detection, gap detection and topic snapshots are manual scripts. SharePoint sync is manual.	E2.5, E26, E28.4 need schedules	**F3**

#	Issue	Impact	Fix (see §4)
D4	`logAudit()` **has only 3 call sites** (org creation, department rename/delete)	E23 and NFR-2	**F2**
D5	**Configuration in code and environment variables** (category list, thresholds, stale days)	NFR-3 (configurability)	**F5**
D6	**No notifications** (beyond auth emails)	E5, E22, E25, E26 need them	**F4**
D7	`Organization.openaiApiKey` stored in plaintext	RFP §6 S5	Encrypt with `src/lib/crypto.js` (Workstream G)

4. Shared Foundations (Build First)

These are used by many requirements. Each has one owner, and other workstreams consume it. Land them **early in Phase 1** so A and B aren't blocked.

F1: Central document-access service (Workstream A)

Why: fixes D1 and is a prerequisite for E2 per-file permissions.

- New module `src/lib/access/documentAccess.js` exporting:
  - `documentAccessSql({ userId, principals, isOrgAdmin, alias })`, the **only** SQL predicate for document visibility;
  - `canAccessDocument(doc, ctx)` (JS equivalent) and `filterAccessibleDocuments(docs, ctx)`;
  - `resolvePrincipals(userId)`, which returns the user's id plus their source-system principals (Entra user id and group ids), cached per session.
- The predicate is the current rule **AND** (the document has no source ACL **OR** an `DocumentAcl` row matches one of the user's principals).
- **Refactor** all copies listed in D1 to call it. No behaviour change in the refactor PR. A parity test asserts identical results before and after on the seeded dataset.

F2: Audit event service (Workstream B)

Why: fixes D4. Required by E19, E22–E25 and NFR-2.

- Extend `AuditLog` usage with a fixed event vocabulary: `document.published | archived | retired | restored | deleted | superseded | version_created | owner_changed | summary_approved | workflow.submitted | workflow.approved | workflow.rejected | review.completed | config.changed | taxonomy.changed`
- Add an optional `documentId` and `entityType` / `entityId` columns (indexed) to `AuditLog` so the document history view can query by entity.
- `logAudit({ orgId, actorUserId, action, entityType, entityId, before, after, metadata })`. It never throws (errors are logged), so it can't break the main action.
- UI: a "History" tab on the document page, and the existing org audit-log view extended with filters.

F3: Scheduled jobs (Workstream A builds it, everyone registers jobs)

Why: fixes D3. Required by E2.5, E26, E28.4, E30 and gap detection.

- **Proposed:** a `ScheduledJob` table (`name`, `orgId?`, `cron`, `enabled`, `lastRunAt`, `nextRunAt`, `lastStatus`, `lastError`) and a scheduler loop in the worker (or a separate `scheduler` process) that claims due jobs with `SELECT … FOR UPDATE SKIP LOCKED` and runs a registered handler.
- Handlers to register: `sharepoint.sync`, `content.stale-scan`, `content.gap-scan`, `topics.snapshot`, `links.check`, `reviews.due-scan`.
- Admin UI: list jobs, enable/disable, edit interval, "run now", last status.
- *Alternative:* AWS EventBridge Scheduler pushing to SQS. It's simpler on AWS, but less portable given NYPA's multi-cloud preference.

F4: Notifications (Workstream B)

Why: fixes D6. Required by E5, E22, E24–E26.

- `Notification` model (`userId`, `orgId`, `type`, `title`, `body`, `link`, `readAt`, `createdAt`) and `NotificationPreference` (per type: in-app, email, digest).
- `notify({ userIds, type, title, body, link })` writes the in-app notification and, per preference, sends email through the existing nodemailer transport.
- UI: a bell icon with an unread count in the header, a notifications list, and a preferences page.

F5: Org configuration service (Workstream B for Phase 1 settings)

Why: fixes D5 and NFR-3.

- `OrgSetting` model (`orgId`, `key`, `value Json`, `updatedBy`, `updatedAt`) plus a typed registry in code (`src/lib/settings/registry.js`) defining each key's schema, default and validation.
- `getSetting(orgId, key)`, cached. Every change goes through F2 (`config.changed`).
- Phase 1 keys: `classification.categoryThreshold`, `classification.departmentThreshold`, `duplicates.threshold`, `stale.rules`, `metadata.requiredFields`, `summary.requireApproval`.

F6: Workflow engine (Workstream B)

Why: one engine for E22 (publishing), E25 (review/validation), E23.6 (decommission approval) and E14 (template assets).

- Models:
  - `WorkflowDefinition` (`orgId`, `name`, `kind` [publish | review | decommission], `scope` [departmentIds, categories/terms], `version`, `active`);
  - `WorkflowStage` (`definitionId`, `order`, `name`, `approverType` [role | owner | steward | users | group], `approverRef`, `rule` [any | all], `slaDays`);
  - `WorkflowInstance` (`definitionId`, `documentId`, `status`, `currentStage`, `startedBy`);
  - `WorkflowTask` (`instanceId`, `stageId`, `assigneeId`, `status` [open | approved | rejected | changes_requested], `comment`, `actedAt`).
- Engine (`src/lib/workflow/engine.js`): `start`, `act(taskId, decision, comment)`, `advance` and `escalate`, all transactional. Every step calls F2 and F4.
- Invariant: **the submitter can never act on a task for their own submission** (team rule: publish gates need a reviewer, not the author).

F7: Unified search service + search events (Workstreams C and D, later phases)

Why: fixes D2. Required by E3, E4, E7, E29 and E30.

- One search entry point, `searchService.search({ query, filters, facets, user })`, wrapping `hybridOrgSearch`. The repository page, the search page, chat retrieval and recommendations all use it.
- `SearchEvent` (`id`, `orgId`, `userId`, `query`, `filters Json`, `resultCount`, `answerShown`, `createdAt`) and `SearchClick` (`searchEventId`, `documentId`, `rank`, `dwellSeconds`).
- *Phase 1 note:* A doesn't depend on F7, but F1 must be used inside `hybridOrgSearch` so permissions apply everywhere.

5. Requirement Implementation Specifications

Format per requirement: **Status** · **Built** (what exists) · **Design** (to build) · **Tests**. Requirements marked Met list their implementation and the regression tests that must keep passing.

5.1 Workstream A: M365 & Connectors (Phase 1, Johurul)

E32: M365 as single source of truth (Partial) *(build first: identity is needed for E2)*

- **Built:** Graph app-only client credentials (`src/lib/msGraph.js`), files downloaded to S3.
- **Design:**
  - **Sign-in (E32.1):** add an Entra ID provider to NextAuth: OIDC (`AzureADProvider`) first, then SAML 2.0 (e.g. via a SAML-to-OAuth bridge such as BoxyHQ SAML Jackson, to evaluate). Per-org SSO configuration stored in `OrgIntegration` (`provider: "entra_sso"`): tenant id, client id, secret or certificate (encrypted with `crypto.js`).
  - **Identity link:** new `UserIdentity` model (`userId`, `provider`, `externalId` [Entra object id], `tenantId`). Users are created on first SSO login (just-in-time) and matched by verified email.
  - **Role mapping:** `OrgSetting` `sso.groupRoleMap` maps Entra group ids to KMS role and departments, applied at each login. Manual roles remain the fallback.
  - **No-copy mode (E32.2):** `OrgSetting` `sharepoint.storeOriginals` (default `false`). When false, the worker downloads into memory for extraction only and doesn't upload to S3. `Document.filePath` stays null.
  - **Deep links (E32.3):** store `externalUrl` (Graph `webUrl`) on `Document`. The UI shows "Open in SharePoint".
  - **On-demand preview (E32.4):** `GET /api/documents/[id]/preview` streams from Graph (`/drives/{d}/items/{i}/content` or the `preview` endpoint) when there's no S3 copy, after checking access with F1.
- **Tests:** SSO login in the test tenant (`nextgenerationinnovation.onmicrosoft.com`). JIT user creation. Group-to-role mapping. No-copy ingestion leaves no S3 object. The deep link resolves.

E2: Unified, permission-respecting search (Partial)

- **Built:** `SharePointConnector` (`src/lib/connectors/sharepoint.js`) on `BaseConnector`, delta cursors per site in `OrgIntegration.scopeConfig`, `SyncRun`, sync API (`api/org/[orgId]/integrations/sharepoint/sync`).
- **Design:**
  - **ACL capture:** new `DocumentAcl` model (`documentId`, `principalType` [user | group | everyone | anonymous-link], `principalId` [Entra id], `source`), indexed on `(documentId)` and `(principalId)`. During sync, call `GET /drives/{d}/items/{i}/permissions` and resolve inherited permissions. Write the full ACL each time an item changes.
  - **Enforcement:** via **F1**. `resolvePrincipals(userId)` returns the user's Entra object id plus their transitive group ids (`/users/{id}/transitiveMemberOf`), cached for 15 minutes.
  - **Permission-only changes:** delta also reports items whose permissions changed, so re-fetch ACLs for them. Also run a periodic full ACL refresh job via F3 as a safety net.
  - **Scheduled sync (E2.5):** register a `sharepoint.sync` handler in F3, with the interval in `OrgSetting`.

    - **Graph permissions** to document for NYPA's admin: `Sites.Selected` (per-site grant) plus whatever is needed to read item permissions, and `GroupMember.Read.All`. To be verified in the test tenant.
    - **Content Server / ServiceNow (E2.8):** new connectors on `BaseConnector` with the same contract (`listChanges(cursor)`, `download(item)`, `getAcl(item)`). ServiceNow first, built against a free developer instance: KB articles via Table API `kb_knowledge`, access via user criteria. Content Server is designed from the OpenText REST API docs and finished when NYPA provides access (awaiting NYPA).
- **Tests:** in the test tenant, a file shared with A but not B (search, chat, citations, recommendations, graph, preview). Revocation propagates after the next sync. Group-based access. A parity test for F1. A load test syncing 5,000 items.

E12: Site pages, wikis and knowledge articles (Gap)

- **Design:** extend `SharePointConnector` with `listPages(siteId)` (`/sites/{id}/pages`, including the SitePages library) and extract the canvas content as HTML, then convert it to text. New `Document.contentType` value `page`, plus `externalUrl`. ACLs captured as in E2. Admin include/exclude per site in `scopeConfig`. ServiceNow KB articles arrive through the E2.8 connector as `contentType: "article"`.
- **Tests:** a modern page and a wiki page are ingested, searchable and citable with a link to the original. Page ACLs are enforced.

5.2 Workstream B: Governance & Lifecycle (Phase 1, Sandeep & Simran)

> Build order inside B: **F2 audit → F5 settings → E23 → E24 → F4 notifications → E19 → E21 → F6 workflow engine → E22 → E25 → E26 → E31**.

E23: Decommission with audit trail (Partial) *(do first)*

- **Built:** `ALLOWED_TRANSITIONS` state machine (`src/app/api/documents/[id]/lifecycle/route.js`). `DELETE /api/documents/[id]` hard-deletes. `POST /api/documents/[id]/replace` re-processes in place.
- **Design:**
    - **Audit:** every lifecycle route calls F2 (`document.archived`, ...), including before/after state.
    - **Supersede:** `Document.supersededById` (self relation) and `supersededAt`. API `POST /api/documents/[id]/supersede { byDocumentId }`. Superseded documents are excluded from default search and answers through a single filter in F1/F7, and their page links to the replacement. E9 "latest version" uses this link.
    - **Versions:** a `DocumentVersion` model (`documentId`, `version`, `filePath`, `contentHash`, `createdBy`, `createdAt`, `changeNote`). `replace` creates a new version row before re-processing, and the old file is kept in S3. UI: a "Versions" tab with view and restore.
    - **Soft delete:** `Document.deletedAt` and `deletedBy`. Excluded everywhere via F1. A purge job (F3) permanently deletes after `OrgSetting retention.deletedDays` (default 30).
- **Tests:** each transition writes exactly one audit row. A superseded document isn't returned by search but is reachable via its link. Version restore. Soft-deleted documents are invisible but purged only after the retention period.

E24: Ownership and stewardship (Partial)

- **Design:**
    - `Document.ownerId` (backfilled from `userId`).
    - `DocumentSteward` (`documentId`, `userId`).
    - A default-owner rule in `OrgSetting` (`ownership.default`: uploader | department designated owner), plus `Department.defaultOwnerId`.
    - APIs: `PATCH /api/documents/[id]/ownership`, and `POST /api/org/[orgId]/ownership/bulk-reassign { fromUserId, toUserId, scope }`.

    - Reassignment moves open tasks (F6), and each change calls F2 (`owner_changed`).
- **Tests:** backfill leaves every document with an owner. Bulk reassignment moves documents and open tasks. Only admins can reassign.

E19: Description with human review (Partial)

- **Built:** `createStructuredSummary` in `worker/summarize.js`. `POST /api/documents/[id]/regenerate`.
- **Design:**
    - Add `Document.summaryStatus` (`pending_review | approved`), `summaryApprovedBy`, `summaryApprovedAt`, `summaryEditedText?`.
    - The worker sets `pending_review`.
    - `PATCH /api/documents/[id]/summary { action: approve | edit, text? }` for owner, steward or department admin.
    - The UI shows an "AI-generated · pending review" badge and an edit/approve control.
    - A "Summaries to review" list for owners.
    - `OrgSetting summary.requireApproval` feeds E21.
- **Tests:** a new document is `pending_review`. Approval writes F2 `summary_approved`. A member who isn't the owner can't approve.

E21: Metadata completeness before publishing (Partial)

- **Design:**
    - `OrgSetting metadata.requiredFields`: a list of rules `{ field, scope: { departmentIds?, categories? } }`.
    - `src/lib/governance/completeness.js` exports `checkCompleteness(document)`, which returns `{ complete, missing[] }`.
    - Called by the lifecycle route and the workflow engine before publishing. Returns 422 with the missing fields.
    - A completeness badge on the document page.
    - A query helper for E28 (incomplete documents).
- **Tests:** publishing is blocked when the owner is missing, and allowed once it's set. The rule is scoped per department.

E22: Configurable publishing workflow (Partial)

- **Design:**
    - F6 with `kind: publish`.
    - Admin UI (Settings → Workflows) to create and edit definitions and stages.
    - On submit, the document lifecycle goes to `in_review` (a new state) and a `WorkflowInstance` starts.
    - Approvers get F4 notifications and a "My tasks" inbox.
    - On final approval, the lifecycle becomes `published`, after the E21 check.
    - **Fallback:** if no definition matches, the current behaviour applies (department admin publishes).
- **Tests:** a 3-stage workflow end to end. Rejection with a comment returns the document to draft. The author can't approve. Every step is audited.

E25: Configurable review/validation workflows (Gap)

- **Design:**
    - `ReviewPolicy` (`orgId`, `scope`, `intervalDays`, `reviewer` [owner | steward | role], `escalateAfterDays`, `escalateTo`).

  - `Document.nextReviewAt` and `lastReviewedAt`.
  - An F3 job `reviews.due-scan` starts F6 instances (`kind: review`) for due documents.
  - Task outcomes:
    - `valid` sets `lastReviewedAt` and recomputes `nextReviewAt`;
    - `update` puts the document in draft and notifies the owner;
    - `retire` starts E23 decommissioning.
  - `ReviewRecord` history per document.
- **Tests:** a due document creates a task for the owner. Overdue reviews escalate. Each outcome updates the dates and audit log correctly.

E26: Stale detection and automated review triggers (Partial)

- **Built:** `scripts/task-8/flag-stale-documents.mjs` (not cited in N days → `lifecycleSuggestion = archived`).
- **Design:** move the logic to `src/lib/governance/stale.js` and register it as F3 job `content.stale-scan`. Rules come from `OrgSetting` `stale.rules` (no update in N days, no views or citations in N days, review overdue, source deleted). A stale document gets `staleSince` plus a review task (E25), with F4 notification. Stale documents carry a badge in search and answers (feeds E10).
- **Tests:** a document matching a rule produces exactly one open review task per scan (no duplicates on later runs).

E31: Governance metrics (Gap)

- **Design:** `GET /api/org/[orgId]/governance/metrics?departmentId&from&to` aggregating from `WorkflowInstance` / `WorkflowTask` (stage counts, approval time, rejection rate), `ReviewPolicy` / `Document.nextReviewAt` (on-time %, overdue), ownership coverage and completeness. Phase 1: an admin page with tables and CSV export. Moves into Workstream D dashboards later.
- **Tests:** the metrics match seeded workflow and review data.

5.3 Workstream C: Taxonomy & Semantic Layer (later phase, owner TBD)

E15: Managed metadata (Gap)

- **Design:**
  - Models: `TermSet` (`orgId`, `name`, `source` [local | external], `externalId`, `readOnly`); `Term` (`termSetId`, `parentId`, `label`, `description`, `status`, `externalId`); `TermSynonym` (`termId`, `label`); `MetadataField` (`orgId`, `name`, `termSetId`, `multi`); `DocumentTerm` (`documentId`, `termId`, `fieldId`, `source` [auto | manual], `confidence`).
  - Migration: the 7 categories become the default "Category" term set, and `Document.category` is backfilled into `DocumentTerm`.
  - `worker/classify.js` receives the active term sets (labels + synonyms + descriptions) in the prompt and maps the result to term ids. Confidence thresholds come from F5.
  - Admin UI: a tree editor (create, nest, rename, deprecate, merge) and a synonym editor. Changes are audited (F2).
  - Auto-discovered topics are offered as candidate terms (approve → term).
- **Tests:** CRUD and hierarchy integrity. A merge re-points tags. Classification produces term ids.

E16: NYPA taxonomy-tool integration (Gap)

- **Design:** an importer `src/lib/taxonomy/import/{skos,csv}.js` creating `TermSet` with `source: external, readOnly: true`. Re-sync diffs by `externalId` (adds, updates, deprecates). An optional connector for NYPA's tool via F3 (awaiting NYPA).
- **Tests:** SKOS import keeps the hierarchy and synonyms. Re-import with changes updates terms without losing document tags.

E3: Ontology-driven search (Partial)

- **Design:** in F7, `expandQueryWithTerms(query)` matches query n-grams to term labels and synonyms (exact plus fuzzy), adds synonyms and narrower terms to the keyword query, and adds a term-match boost to the hybrid score (`w_keyword`, `w_vector`, `w_term`, `w_graph` from F5). An offline evaluation set (`scripts/ekp/search-eval/`) measures nDCG@10 before and after.
- **Tests:** a synonym query returns tagged documents. Evaluation score improves on the agreed query set.

E4: Faceted navigation (Partial)

- **Design:** F7 returns `facets: [{ fieldId, buckets: [{ termId, label, count, children }] }]`, computed with the F1 predicate over `DocumentTerm`. A facet panel component shared by the repository and search pages. Admins choose facet fields (F5).
- **Tests:** facet counts equal accessible-document counts. Parent/child narrowing.

5.4 Workstream D: Analytics & Dashboards (later phase, owner TBD)

Common approach: an `/org/[orgId]/analytics` section with tabs (Content health, Usage, Search quality, Governance). APIs under `api/org/[orgId]/analytics/*`, permission-scoped (department admins see their departments). CSV export. Read-only reporting endpoints, plus a documented Power BI connection (RFP §6 A5). Heavy aggregations are precomputed nightly by an F3 job into `AnalyticsDaily` (`orgId`, `departmentId`, `date`, `metric`, `value`).

E28: Content health dashboard (Partial)

- **Built:** `api/org/[orgId]/health` (confidence, conflicts, gaps).
- **Design:** recency buckets from `updatedAt` and `staleSince`; ownership coverage from E24; review status from E25; duplicates from `DocumentDuplicate`; completeness from E21. **Broken links:** an F3 job `links.check` extracts URLs from document content into `DocumentLink` (`documentId`, `url`, `status`, `checkedAt`), checks them with HEAD/GET plus a timeout, and flags synced items whose source returns 404.
- **Tests:** each widget's count matches the seeded data. A broken link is detected.

E29: Usage and adoption dashboard (Partial)

- **Built:** `ChatAuditLog`, `DocumentInteraction` (`api/documents/[id]/interactions`, dwell capped at 1,200 s).
- **Design:** F7 `SearchEvent` / `SearchClick`. CTR = searches with ≥ 1 click ÷ searches. Time-to-answer = first click or answer-render time − search time. Bounce = searches with no click and no follow-up within 60 s. Top queries are normalized (lowercase, trimmed). Active users from events.
- **Tests:** synthetic events produce the expected metrics.

E30: Search quality metrics (Partial)

- **Built:** `KnowledgeGap` (zero-citation and low-confidence questions), dwell.
- **Design:** no-results list from `SearchEvent.resultCount = 0`. CTR@1/3/10. Reformulation rate (a new query within 60 s by the same user). Answer feedback from E10.4. Gaps link to the triggering queries (`KnowledgeGap.queryIds`). The gap scan becomes an F3 job.
- **Tests:** a zero-result query appears in the no-results list. The metrics are correct on fixtures.

5.5 Workstream E: Content Coverage (later phase, owner TBD)

E11: More formats (Partial)

- **Built:** `worker/index.js` extraction branches (PDF + OCR, DOCX, spreadsheets, TXT). Upload allow-list in `UploadToRepositoryModal.jsx`.

- **Design:**
  - **PPTX:** unzip the slide XML (`ppt/slides/slideN.xml`, `ppt/notesSlides/`), one chunk per slide with `metadata.slide`. Citations show "slide N".
  - **Images:** Tesseract OCR (already a dependency) plus a vision-model caption. `metadata.kind = image`.
  - **Audio/video:** transcription (Whisper API, or a US-region equivalent for S9) with segment timestamps, chunked by time. Citations link to `?t=`. Large media are processed as separate jobs with progress.
  - Extend the upload allow-list and the SharePoint mime-type filter.
- **Tests:** a fixture per format is ingested and cited with the correct slide or timestamp.

E13: Previews and thumbnails (Partial)

- **Design:** the worker generates thumbnails (PDF first page via `pdfjs-dist`, PPTX rendered or the first slide's image, image downscale, video frame at 1 s) into S3 `thumbnails/{docId}.jpg`, stored as `Document.thumbnailPath`. Preview components for images, slides and video. Previews for M365 items come from Graph when there's no copy (E32.4).
- **Tests:** thumbnails exist for each visual format, and the UI renders them.

E14: Asset templates (Gap)

- **Design:** `AssetTemplate` (`orgId`, `name`, `version`, `sections Json` [title, guidance, required], `requiredFields`). A rich-text editor page "New from template" creates a `Document` with `sourceProvider: "authored"` and `contentType: "asset"`, whose content is stored as structured JSON plus rendered text. It enters F6 publishing (E22) and is ingested like any document. Template versioning keeps existing assets unchanged.
- **Tests:** create from template, then publish through the workflow, and the asset is searchable. A template change doesn't alter existing assets.

5.6 Workstream F: Personalization & Experience (later phase, owner TBD)

E1: Customizable landing page (Partial)

- **Design:** `LandingPageConfig` in F5 (`landing.page`: welcome text, featured item ids, quick links, ordered widget list, per-department variants) and `OrgSetting` `branding` (logo S3 key, primary colour). Featured items are filtered with F1 at render time. A new `/org/[orgId]` landing route replaces the redirect.
- **Tests:** the configuration renders without a deploy. Inaccessible featured items are hidden.

E5: Personal homepage, topic hubs, saved searches, watchlists (Partial)

- **Design:**
  - `SavedSearch` (`userId`, `orgId`, `name`, `query`, `filters`, `notify`).
  - `Watch` (`userId`, `targetType` [document | topic | term | savedSearch], `targetId`).
  - An F3 job evaluates watches and saved searches against new or updated documents, then calls F4.
  - Personal home `/org/[orgId]/home`: watch updates, saved searches, recommendations (E6), recent views, my tasks (F6).
  - Topic hub `/org/[orgId]/topics/[id]`: top documents, recent changes, experts (Rank 9), related decisions and lessons, related topics (graph).
- **Tests:** a new matching document notifies the watcher. Hub content is permission-filtered.

E7: Answers alongside search results (Met, polish)

- **Design:** a unified search page `/org/[orgId]/search` (via F7). For question-like queries, it calls a non-streaming answer variant of the chat pipeline and shows an answer card with citations above the result list, with "Continue in chat".
- **Tests:** a question-style query shows the answer card and results. A keyword query shows results only.

E10: Confidence and explainability (Partial)

- **Built:** `computeConfidence(chunks)` in the chat route.
- **Design:** numeric `confidence` (0–1) from a documented formula (top-k similarity, keyword agreement, number of independent sources, conflict penalty, staleness penalty). An `explanation` object stored on `OrgMessage` (sources with match scores, conflicts from `DocumentConflict`, stale flags from E26). A "Why this answer" panel. thumbs up / thumbs down feedback into an `AnswerFeedback` model (feeds E30).
- **Tests:** the confidence formula unit-tested on fixtures. The explanation lists exactly the cited sources.

5.7 Completed requirements: regression guard

#	Regression tests that must pass on every PR touching the area
E6	`npm run task11:test` (recommendation ranking and static)
E8, E9	Chat route permission tests. `npm run alpha:combined-test`. **New with F1:** cross-user citation leak test.
E17	`npm run task13:test`, `npm run task14:test` (relationships and graph)
E18	Upload-flow integration check (manual + ingest API test)
E20	`npm run task8:test`, `task8:test:integration` (classification)
E27	Duplicate-detection cases in the task-8 suite

6. Data Model Changes (Summary)

Model / field	Change	Requirement(s)	Phase
`DocumentAcl`	new	E2, E12	1
`UserIdentity`	new	E32	1
`Document.externalUrl`, `contentType`	new fields	E32, E12	1
`AuditLog.entityType`, `entityId`	new fields + index	F2, E23	1
`OrgSetting`	new	F5	1
`ScheduledJob`	new	F3	1
`Notification`, `NotificationPreference`	new	F4	1
`Document.supersededById`, `supersededAt`, `deletedAt`, `deletedBy`	new fields	E23	1
`DocumentVersion`	new	E23	1
`Document.ownerId`, `DocumentSteward`, `Department.defaultOwnerId`	new	E24	1
`Document.summaryStatus`, `summaryApprovedBy`, `summaryApprovedAt`	new fields	E19	1
`WorkflowDefinition`, `WorkflowStage`, `WorkflowInstance`, `WorkflowTask`	new	F6, E22	1
`ReviewPolicy`, `ReviewRecord`, `Document.nextReviewAt`, `lastReviewedAt`, `staleSince`	new	E25, E26	1
`TermSet`, `Term`, `TermSynonym`, `MetadataField`, `DocumentTerm`	new	E15, E16, E3, E4	later

Model / field	Change	Requirement(s)	Phase
`SearchEvent`, `SearchClick`, `AnalyticsDaily`, `DocumentLink`, `AnswerFeedback`	new	E28–E30, E10	later
`Document.thumbnailPath`, `AssetTemplate`	new	E13, E14	later
`SavedSearch`, `Watch`	new	E5	later

Migration rules (mandatory): never run `prisma migrate reset`. Never pass the live database as the shadow database. Generate SQL with `prisma migrate diff --from-url … --to-schema-datamodel prisma/schema.prisma --script` and apply it with `prisma migrate deploy`. One migration per PR. A and B both add `Document` fields in Phase 1, so agree field names in the requirements review and merge schema PRs one at a time.

7. Phase 1 Build Sequence

Week	Workstream A (Johurul)	Workstream B (Sandeep & Simran)
1	Requirements doc A. **F1** refactor (no behaviour change) + parity test	Requirements doc B. **F2** audit service. **F5** settings
2	**E32** Entra sign-in (OIDC), `UserIdentity`, group → role map	**E23** audit wiring, supersede, versions, soft delete. **E24** ownership
3	**F3** scheduler. **E2** `DocumentAcl` capture + enforcement via F1	**F4** notifications. **E19** summary review. **E21** completeness
4	**E2** scheduled sync, permission-change handling, load test	**F6** workflow engine. **E22** publishing workflow
5	**E12** site pages. **E32** deep links, no-copy mode, on-demand preview	**E25** review policies. **E26** stale scan as a scheduled job
6	SAML 2.0 support. ServiceNow connector (developer instance)	**E31** governance metrics. Hardening, end-to-end tests
Later	Content Server connector (awaiting NYPA access)	—

Dependencies across lanes: F1 (A) is needed by B's soft-delete and supersede filters, so A delivers F1 in week 1. F3 (A) is needed by E26 (B) in week 5. F2 and F4 (B) are used by A for audit and notifications from week 2.

8. Testing Strategy

Level	What	Where
Unit / logic	Policies and pure functions (access predicate builder, completeness, confidence, workflow transitions)	`scripts/ekp/<ws>/*.logic.test.mjs`
Integration	API + database against a seeded test org (roles: org admin, department admin, member, guest)	`scripts/ekp/<ws>/*.integration.test.mjs`
Permission	For every new endpoint and every search path: member and guest can't see restricted data. Cross-user citation leak test.	Required for each PR
Parity	F1 refactor returns identical results before and after	`scripts/ekp/f1-access-parity.test.mjs`
Connector	SharePoint test tenant (per-file sharing scenarios), ServiceNow developer instance	Manual checklist + scripted checks
Performance	Sync and ingest 500 documents in 30 min. Search p95 ≤ 2 s on a 10k-document dataset *(proposed)*.	`scripts/ekp/perf/`
Regression	§5.7 suites	Run before every PR in the affected area

Add an `npm run ekp:test` script that runs all `scripts/ekp/**` suites.

9. Risks

Risk	Mitigation
The F1 refactor changes behaviour and causes data leaks or over-restriction	No-behaviour-change PR first, with a parity test, then add ACLs in a separate PR
Graph permission reads need broader app permissions than Sites.Selected	Verify in the test tenant in week 3, and document the exact permission set for NYPA's admin
Group expansion is slow for users in many groups	Cache transitive groups per session (15 min), and index `DocumentAcl.principalId`
Schema conflicts between A and B on `Document`	Agree fields in week 1, one migration per PR, merge schema PRs one at a time
Workflow engine scope creep	Phase 1 supports linear stages with any/all approval only. No parallel branches.
NYPA answers change assumptions (e.g. on-premises SharePoint, SAML only)	Connector interface and SSO configuration are pluggable. Revisit within 2 days of the answers.